

// Jivops

Traefik

Guía completa — 20 lecciones

Fundamentos · Middlewares y servicios · TLS y seguridad · Kubernetes y operación

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es Traefik
- Lección 02 — Configuración estática y dinámica
- Lección 03 — EntryPoints, routers y services
- Lección 04 — Providers
- Lección 05 — Reglas de enrutado

Módulo 2: Middlewares y servicios

- Lección 06 — Middlewares y cadenas
- Lección 07 — redirectScheme, stripPrefix y headers
- Lección 08 — Autenticación: basicAuth y forwardAuth
- Lección 09 — rateLimit, circuitBreaker y retry
- Lección 10 — Balanceo, sticky y health checks

Módulo 3: TLS y seguridad

- Lección 11 — HTTPS y certificatesResolvers
- Lección 12 — Desafíos ACME
- Lección 13 — Opciones TLS
- Lección 14 — Seguridad del dashboard y de la socket de Docker
- Lección 15 — Allowlist de IP y cabeceras de seguridad

Módulo 4: Kubernetes y operación

- Lección 16 — Traefik en Kubernetes: IngressRoute
- Lección 17 — Gateway API
- Lección 18 — Observabilidad
- Lección 19 — Troubleshooting y actualizaciones
- Lección 20 — Proyecto final: Traefik listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es Traefik

Un proxy inverso y balanceador pensado para entornos dinámicos: descubre servicios automáticamente desde proveedores como Docker o Kubernetes y actualiza sus rutas en caliente, sin reinicios, a partir de etiquetas o recursos del orquestador.

Detecta servicios nuevos o eliminados y actualiza las rutas sin reiniciar ni editar ficheros a mano.

Lección 02 — Configuración estática y dinámica

La estática (fichero, flags o variables de entorno) define entryPoints, providers y resolvers y requiere reiniciar Traefik al cambiarla; la dinámica define routers, services y middlewares y se recarga en caliente cuando cambia en el provider.

El puerto (entryPoint) en la configuración estática; la ruta hacia el servicio (router y service) en la dinámica.

Lección 03 — EntryPoints, routers y services

El entryPoint es el puerto por el que entra el tráfico, el router decide con reglas (host, ruta) a qué servicio va cada petición y el service define los servidores de destino y cómo se balancea entre ellos; los middlewares se aplican entre router y service.

Entra por un entryPoint, un router la hace coincidir con una regla, pasa por los middlewares y llega a un service que la envía a un servidor.

Lección 04 — Providers

La fuente de la configuración dinámica: Traefik la lee de Docker (etiquetas), Kubernetes (Ingress, IngressRoute, Gateway API), ficheros u otros orígenes, y reacciona a sus cambios; se habilitan en la configuración estática.

Para que solo se expongan los contenedores marcados explícitamente con `traefik.enable=true` y no todos los del host.

Lección 05 — Reglas de enrutado

Con expresiones como `Host(`ejemplo.com`)`, `PathPrefix(`/api`)`, `Headers` o combinaciones con `&&` y `||`; si varias reglas coinciden, gana la de mayor prioridad, que por defecto depende de la longitud de la regla y puede fijarse con `priority`.

Con dos routers: uno con `Host` y `PathPrefix(`/api`)` hacia el servicio de la API y otro solo con `Host` hacia el resto, ajustando `priority` si hace falta.

Módulo 2: Middlewares y servicios

Lección 06 — Middlewares y cadenas

Un componente que modifica o filtra la petición o la respuesta entre el router y el servicio (autenticación, redirecciones, límites, cabeceras); se asocian al router y pueden encadenarse en el orden indicado.

Porque se ejecutan en orden y el resultado depende de ello, por ejemplo la autenticación debe ir antes que el reenvío.

Lección 07 — `redirectScheme`, `stripPrefix` y `headers`

`redirectScheme` redirige de HTTP a HTTPS, `stripPrefix` elimina un prefijo de la ruta antes de enviarla al servicio y `headers` añade o modifica cabeceras (por ejemplo de seguridad) en la petición o la respuesta.

Cuando la aplicación está en la raíz y no espera el prefijo con el que se expone (por ejemplo `/api`).

Lección 08 — Autenticación: `basicAuth` y `forwardAuth`

`basicAuth` valida usuario y contraseña definidos en Traefik (siempre sobre HTTPS), mientras que `forwardAuth` envía la petición a un servicio externo de autenticación y deja pasar o bloquea según su respuesta, lo que permite integrar SSO u OAuth.

Porque las credenciales viajan solo codificadas en base64 y en HTTP pueden interceptarse.

Lección 09 — `rateLimit`, `circuitBreaker` y `retry`

`rateLimit` limita las peticiones por segundo (por origen), `circuitBreaker` deja de enviar tráfico a un servicio degradado según errores o latencia y `retry` reintenta peticiones fallidas, con cuidado de no duplicar operaciones no idempotentes.

Deja de enviar tráfico al servicio degradado durante un tiempo, protegiéndolo de la saturación y devolviendo errores rápidos.

Lección 10 — Balanceo, `sticky` y `health checks`

Reparto por turnos entre servidores (con pesos si se usa un servicio ponderado), sesiones `sticky` mediante cookie, `health checks` para retirar servidores no sanos y `mirroring` para copiar tráfico a otro servicio, por ejemplo para pruebas.

Para probar una versión nueva con tráfico real copiado, sin afectar a las respuestas que reciben los usuarios.

Módulo 3: TLS y seguridad

Lección 11 — HTTPS y certificatesResolvers

Con un certificatesResolver ACME (por ejemplo Let's Encrypt) definido en la configuración estática, que guarda los certificados en un fichero (acme.json) con permisos restringidos, y con el router indicando `tls.certresolver` para que se emitan y renueven solos.

Para no perder los certificados al reiniciar y evitar pedirlos de nuevo, lo que podría agotar los límites de emisión de Let's Encrypt.

Lección 12 — Desafíos ACME

El desafío DNS-01, porque exige demostrar el control del dominio creando un registro TXT mediante la API del proveedor DNS; los desafíos HTTP-01 y TLS-ALPN-01 no pueden emitir certificados wildcard.

Credenciales de la API de tu proveedor DNS para crear automáticamente el registro TXT de validación.

Lección 13 — Opciones TLS

Las versiones mínima y máxima de TLS, los cifrados permitidos, la negociación ALPN y la autenticación mutua con certificados de cliente (mTLS), aplicables a los routers que las referencian.

Para aceptar solo clientes que presenten un certificado válido, por ejemplo en APIs internas o entre servicios.

Lección 14 — Seguridad del dashboard y de la socket de Docker

No exponer el dashboard sin autenticación ni con `api.insecure` en producción, protegerlo con un router, TLS y middlewares, y no dar acceso directo de escritura a `/var/run/docker.sock` (montarlo en solo lectura o usar un proxy de la socket).

Porque da control sobre Docker y, por tanto, casi sobre el host: se podrían crear contenedores privilegiados.

Lección 15 — Allowlist de IP y cabeceras de seguridad

Con el middleware `ipAllowList` (en v2 se llamaba `ipWhiteList`), indicando los rangos permitidos, y combinándolo con cabeceras de seguridad (HSTS, X-Content-Type-Options) mediante el middleware `headers`.

Que Traefik vea la IP del balanceador y no la del cliente; hay que confiar en X-Forwarded-For desde IP de confianza (`forwardedHeaders.trustedIPs`).

Módulo 4: Kubernetes y operación

Lección 16 — Traefik en Kubernetes: IngressRoute

Es un recurso propio (CRD) de Traefik que expresa reglas, middlewares, servicios ponderados y opciones TLS de forma más completa que el Ingress estándar, que depende de anotaciones; se instala normalmente con su chart de Helm.

Es más expresivo y tipado: middlewares y TLS son recursos con esquema, en lugar de anotaciones difíciles de validar.

Lección 17 — Gateway API

Una API estándar de Kubernetes (Gateway, HTTPRoute, etc.) para configurar el enrutado de forma portable entre implementaciones; Traefik la soporta como provider, además de Ingress e IngressRoute.

Portabilidad: puedes cambiar de implementación sin reescribir las rutas y evitas depender de anotaciones específicas.

Lección 18 — Observabilidad

Métricas (Prometheus y otros), registros de acceso y de aplicación en JSON o texto y trazas distribuidas con OpenTelemetry, además del dashboard para ver routers, services y middlewares.

Aumento de errores 5xx y de latencia, o servicios caídos que no tienen servidores disponibles.

Lección 19 — Troubleshooting y actualizaciones

Revisando en el dashboard o la API que el router existe y su regla coincide, el entryPoint, que el provider ve el servicio (por ejemplo traefik.enable y la red), los logs de Traefik con nivel DEBUG y los cambios de sintaxis entre versiones mayores.

Leer la guía de migración (por los cambios de sintaxis y opciones) y probar en un entorno de pruebas.

Lección 20 — Proyecto final: Traefik listo para producción

Varias réplicas con HTTPS y certificados automáticos persistidos, redirección de HTTP a HTTPS, dashboard protegido, socket de Docker restringida o IngressRoute en Kubernetes, middlewares de seguridad y rateLimit, health checks, métricas con alertas y versión fijada.

HTTPS con certificados automáticos persistidos, dashboard y socket protegidos y monitorización con alertas.